

# C++ Programming

Day 2: 조건 분기와 반복 제어

if-else, switch, do-while, break, continue, 난수

## Contents

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| <b>1</b>  | <b>학습 목표</b>                            | <b>2</b>  |
| <b>2</b>  | <b>두 가지 경우 선택: if--else</b>             | <b>2</b>  |
| 2.1       | 예제: 성인 여부 판별                            | 2         |
| 2.2       | 비교 연산자와 대입 연산자                          | 3         |
| <b>3</b>  | <b>여러 조건 처리: else if</b>                | <b>3</b>  |
| 3.1       | 예제: 학점 판별                               | 4         |
| 3.2       | 조건은 위에서 아래로 검사된다                        | 4         |
| <b>4</b>  | <b>조건문 안의 조건문: Nested if</b>            | <b>5</b>  |
| 4.1       | 예제: 단계적 로그인 검사                          | 5         |
| <b>5</b>  | <b>하나의 값으로 여러 경우 선택: switch</b>         | <b>6</b>  |
| 5.1       | 예제: 메뉴 선택                               | 6         |
| 5.2       | break와 fall-through                     | 7         |
| 5.3       | if--else와 switch 비교                     | 8         |
| <b>6</b>  | <b>최소 한 번 실행: do--while</b>             | <b>8</b>  |
| 6.1       | 입력 검증                                   | 8         |
| <b>7</b>  | <b>반복 종료: break</b>                     | <b>9</b>  |
| 7.1       | 무한 반복과 break                            | 9         |
| <b>8</b>  | <b>현재 반복 건너뛰기: continue</b>             | <b>10</b> |
| 8.1       | break와 continue 비교                      | 10        |
| 8.2       | 예외를 먼저 처리하기                             | 10        |
| <b>9</b>  | <b>난수 생성의 기본 원리</b>                     | <b>11</b> |
| 9.1       | rand()                                  | 11        |
| 9.2       | seed와 srand()                           | 11        |
| <b>10</b> | <b>원하는 범위의 난수 만들기</b>                   | <b>12</b> |
| 10.1      | 일반적인 범위 공식                              | 12        |
| <b>11</b> | <b>종합 실습: 숫자 맞추기 게임</b>                 | <b>12</b> |
| 11.1      | 완성 코드: practice/p01_number_guessing.cpp | 13        |
| 11.2      | 실행 흐름 분석                                | 13        |
| <b>12</b> | <b>프로젝트 파일 구성</b>                       | <b>14</b> |
| <b>13</b> | <b>핵심 정리</b>                            | <b>14</b> |
| <b>14</b> | <b>체크 질문</b>                            | <b>15</b> |
| <b>15</b> | <b>추가 연습 문제</b>                         | <b>15</b> |

## 1. 학습 목표

### 학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- if--else와 else if를 이용한 다중 조건 분기
- 중첩 조건문을 이용한 단계적 판단
- switch문과 break의 역할
- while문과 do--while문의 차이
- break, continue를 이용한 반복 흐름 제어
- rand(), srand(), time()을 이용한 난수 생성
- 조건문, 반복문, 난수를 결합한 숫자 맞추기 게임 구현

Day 1에서는 프로그램의 기본 구조와 기본적인 조건문 및 반복문을 학습했다. Day 2에서는 이미 배운 문법을 더 정교하게 사용하여 프로그램의 실행 흐름을 설계한다. 중요한 것은 문법을 외우는 것이 아니라, 어떤 조건에서 어느 코드가 실행되고 반복이 언제 계속되거나 종료되는지를 정확히 이해하는 것이다.

## 2. 두 가지 경우 선택: if--else

if문은 조건이 참일 때만 코드를 실행한다. 그러나 실제 프로그램에서는 조건이 참일 때와 거짓일 때 서로 다른 작업을 수행해야 하는 경우가 많다. 이때 if--else를 사용한다.

```

1 if (condition)
2 {
3     // condition - 128) * 64 + (' - 128) 뽕 true - 128) * 64 + (' - 128) 뽕
4     - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 퀘
5 }
6 else
7 {
8     // condition - 128) * 64 + (' - 128) 뽕 false - 128) * 64 + (' - 128) 뽕
9     - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 퀘
10 }
```

### 2.1. 예제: 성인 여부 판별

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int age;
8
9     cout << "Age: ";
10    cin >> age;
11
12    if (age >= 20)
13    {
14        cout << "Adult" << endl;
```

```

15 }
16 else
17 {
18     cout << "Minor" << endl;
19 }
20
21 return 0;
22 }

```

입력이 23이면 `age >= 20`이 참이므로 `Adult`가 출력된다. 입력이 17이면 조건이 거짓이므로 `Minor`가 출력된다.

### 핵심 포인트

`if--else`에서는 두 블록 중 정확히 하나만 실행된다. 서로 동시에 성립할 수 없는 두 경우를 처리할 때 적합하다.

## 2.2. 비교 연산자와 대입 연산자

다음 두 표현은 의미가 완전히 다르다.

```

1 number = 10; // - 128) * 64 + (' - 128) 끝 - 128) * 64 + (' - 128) 뽀
2 number == 10; // - 128) * 64 + (' - 128) 끝 - 128) * 64 + (' - 128)

```

`=`는 오른쪽 값을 왼쪽 변수에 저장하는 대입 연산자이고, `==`는 두 값이 같은지 검사하는 비교 연산자이다.

```

1 if (number == 10)
2 {
3     cout << "The number is 10." << endl;
4 }

```

조건식에서 실수로 `number = 10`을 쓰면 10이 변수에 대입되고, 0이 아닌 값이 참으로 평가되어 의도와 다르게 작동할 수 있다.

### 실습

파일: 01\_if\_else.cpp

정수 하나를 입력받아 0 이상이면 Non-negative, 0보다 작으면 Negative를 출력하라.

## 3. 여러 조건 처리: else if

두 가지보다 많은 경우를 처리하려면 `else if`를 사용한다.

```

1 if (condition1)
2 {
3     // condition1 - 128) * 64 + (' - 128) 뽀 true
4 }
5 else if (condition2)
6 {
7     // condition1 - 128) * 64 + (' - 128) 뽀
8         false - 128) * 64 + (' - 128) 뽀 - 128) * 64 + (' - 128)
9         condition2 - 128) * 64 + (' - 128) 끝 true
10 }
11 else
12 {

```

```

11 // - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕
    - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 꿀
    - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) - 128) * 64 + (' - 128) 뽕
    false
12 }

```

### 3.1. 예제: 학점 판별

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int score;
8
9     cout << "Score: ";
10    cin >> score;
11
12    if (score >= 90)
13    {
14        cout << "Grade A" << endl;
15    }
16    else if (score >= 80)
17    {
18        cout << "Grade B" << endl;
19    }
20    else if (score >= 70)
21    {
22        cout << "Grade C" << endl;
23    }
24    else if (score >= 60)
25    {
26        cout << "Grade D" << endl;
27    }
28    else
29    {
30        cout << "Grade F" << endl;
31    }
32
33    return 0;
34 }

```

### 3.2. 조건은 위에서 아래로 검사된다

`else if` 구조에서는 조건을 위에서부터 차례로 검사한다. 처음으로 참이 된 블록 하나만 실행되고 나머지는 검사하지 않는다.

따라서 다음처럼 낮은 기준을 먼저 쓰면 잘못된 결과가 나온다.

```

1 if (score >= 60)
2 {
3     cout << "Grade D" << endl;
4 }
5 else if (score >= 90)
6 {
7     cout << "Grade A" << endl;
8 }

```

점수가 95여도 첫 번째 조건이 이미 참이므로 Grade D가 출력된다. 범위가 겹치는 조건은 일반적으로 더 제한적인 조건부터 작성해야 한다.

### 핵심 포인트

`else if`는 위에서부터 검사하고, 처음 참인 블록 하나만 실행한다. 따라서 조건의 순서가 프로그램의 결과를 바꿀 수 있다.

### 실습

**파일:** 02\_else\_if.cpp

정수 하나를 입력받아 양수이면 Positive, 0이면 Zero, 음수이면 Negative를 출력하라.

## 4. 조건문 안의 조건문: Nested if

조건문 내부에 또 다른 조건문을 작성하는 것을 중첩 조건문이라고 한다.

```

1 if (condition1)
2 {
3     if (condition2)
4     {
5         // - 128) * 64 + (' - 128) 끝
5         - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) - 128) * 64 + (' - 128) 뽕
5         - 128) * 64 + (' - 128) 끝 - 128) * 64 + (' - 128) 끝
5         - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕
5         - 128) * 64 + (' - 128) 끝
5         - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 켜
6     }
7 }

```

### 4.1. 예제: 단계적 로그인 검사

```

1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main()
7 {
8     string password;
9     bool isAdmin = true;
10
11     cout << "Password: ";
12     cin >> password;
13
14     if (password == "cpp123")
15     {
16         cout << "Login successful." << endl;
17
18         if (isAdmin)
19         {
20             cout << "Administrator mode." << endl;
21         }
22     }
23     else
24     {

```

```

25     cout << "Login failed." << endl;
26 }
27
28 return 0;
29 }

```

안쪽의 if (isAdmin)은 바깥쪽 비밀번호 검사가 성공했을 때만 평가된다. 두 조건을 항상 동시에 검사해도 된다면 논리 연산자 &&를 사용할 수 있다.

```

1 if (password == "cpp123" && isAdmin)
2 {
3     cout << "Administrator mode." << endl;
4 }

```

단계별로 다른 메시지를 출력해야 한다면 중첩 조건문이 더 자연스럽다. 다만 중첩이 지나치게 깊으면 코드 흐름을 이해하기 어려워지므로 주의한다.

### 실습

**파일:** 03\_nested\_if.cpp

정수 하나를 입력받아 먼저 양수인지 검사하라. 양수라면 다시 짝수인지 홀수인지 검사하고, 0 또는 음수라면 Not a positive number를 출력하라.

## 5. 하나의 값으로 여러 경우 선택: switch

하나의 변수나 표현식이 특정 값과 일치하는지 여러 번 비교할 때 switch문을 사용할 수 있다.

```

1 switch (expression)
2 {
3     case value1:
4         // expression == value1
5         break;
6
7     case value2:
8         // expression == value2
9         break;
10
11    default:
12        //
13        - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 켜 - 128) * 6
14        case - 128) * 64 + (' - 128)
15        - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕
16        - 128) * 64 + (' - 128) 꿀
17        break;
18 }

```

### 5.1. 예제: 메뉴 선택

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int menu;
8
9     cout << "1. Start" << endl;

```

```
10  cout << "2. Settings" << endl;
11  cout << "3. Exit" << endl;
12  cout << "Select: ";
13  cin >> menu;
14
15  switch (menu)
16  {
17      case 1:
18          cout << "Game started." << endl;
19          break;
20
21      case 2:
22          cout << "Settings opened." << endl;
23          break;
24
25      case 3:
26          cout << "Program ended." << endl;
27          break;
28
29      default:
30          cout << "Invalid menu." << endl;
31          break;
32  }
33
34  return 0;
35 }
```

## 5.2. break와 fall-through

break는 현재 switch문을 즉시 종료한다. break가 없으면 일치한 case 아래의 코드가 다음 case까지 연속해서 실행될 수 있다.

```
1  int number = 1;
2
3  switch (number)
4  {
5      case 1:
6          cout << "One" << endl;
7      case 2:
8          cout << "Two" << endl;
9      case 3:
10         cout << "Three" << endl;
11 }
```

출력 결과:

One  
Two  
Three

이 현상을 fall-through라고 한다. 의도적으로 사용할 수도 있지만, 초반에는 각 case의 마지막에 break를 작성하는 것이 안전하다.



### 5.3. if--else와 switch 비교

| 상황               | 적합한 문법   |
|------------------|----------|
| 값의 범위를 검사        | if--else |
| 두 조건을 동시에 검사     | if--else |
| 하나의 값과 여러 상수를 비교 | switch   |
| 메뉴 번호 선택         | switch   |
| 복잡한 논리 조건        | if--else |

#### 실습

파일: 04\_switch.cpp

1부터 7 사이의 숫자를 입력받아 해당 요일의 영어 이름을 출력하라. 그 외의 값은 Invalid day 로 처리한다.

## 6. 최소 한 번 실행: do--while

while문은 조건을 먼저 검사한다. 따라서 처음부터 조건이 거짓이면 본문이 한 번도 실행되지 않는다.

```

1 int number = 10;
2
3 while (number < 5)
4 {
5     cout << number << endl;
6 }
```

반면 do--while문은 본문을 먼저 실행한 뒤 조건을 검사한다.

```

1 do
2 {
3     // - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 켜
4     - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 꿀
5 while (condition);
```

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int number = 10;
8
9     do
10    {
11        cout << number << endl;
12    }
13    while (number < 5);
14
15    return 0;
16 }
```

조건은 거짓이지만 본문이 먼저 실행되므로 10이 한 번 출력된다.

### 6.1. 입력 검증

사용자의 입력을 먼저 받아야 조건을 검사할 수 있으므로, 입력 검증에는 do--while문이 자연스럽다.

```

1 int number;
2
3 do
4 {
5     cout << "Enter a number from 1 to 10: ";
6     cin >> number;
7 }
8 while (number < 1 || number > 10);
9
10 cout << "Accepted: " << number << endl;

```

### 핵심 포인트

do--while문의 마지막 while (condition) 뒤에는 반드시 세미콜론이 필요하다.

### 실습

파일: 05.do\_while.cpp

사용자가 0을 입력할 때까지 정수를 계속 입력받아 각 값을 출력하라. 0을 입력하면 반복을 종료한다.

## 7. 반복 종료: break

break는 현재 실행 중인 가장 가까운 반복문이나 switch문을 즉시 종료한다.

```

1 for (int number = 1; number <= 10; number++)
2 {
3     if (number == 5)
4     {
5         break;
6     }
7
8     cout << number << endl;
9 }

```

출력 결과:

```

1
2
3
4

```

### 7.1. 무한 반복과 break

반복 횟수를 미리 정하기 어려울 때는 무한 반복을 만든 뒤 특정 조건에서 종료할 수 있다.

```

1 while (true)
2 {
3     int number;
4
5     cout << "Enter 0 to quit: ";
6     cin >> number;
7
8     if (number == 0)
9     {
10        break;

```

```

11 }
12
13 cout << "You entered " << number << endl;
14 }

```

중첩 반복문 안의 `break`는 가장 가까운 반복문 하나만 종료한다.

### 실습

**파일:** 06\_break.cpp

1부터 100까지 차례로 검사하다가 처음으로 7의 배수이면서 20보다 큰 수를 찾으면 출력하고 반복을 종료하라.

## 8. 현재 반복 건너뛰기: `continue`

`continue`는 반복문 전체를 종료하지 않는다. 현재 반복에서 남은 코드를 건너뛰고 다음 반복으로 이동한다.

```

1 for (int number = 1; number <= 10; number++)
2 {
3     if (number % 2 == 0)
4     {
5         continue;
6     }
7
8     cout << number << endl;
9 }

```

출력 결과:

```

1
3
5
7
9

```

### 8.1. `break`와 `continue` 비교

| 문법                    | 동작                              |
|-----------------------|---------------------------------|
| <code>break</code>    | 반복문 전체를 즉시 종료한다.                |
| <code>continue</code> | 현재 반복의 남은 부분만 건너뛰고 다음 반복을 수행한다. |

### 8.2. 예외를 먼저 처리하기

```

1 for (int count = 0; count < 5; count++)
2 {
3     int number;
4     cin >> number;
5
6     if (number < 0)
7     {
8         cout << "Negative numbers are ignored." << endl;
9         continue;
10    }
11 }

```

```

12     cout << "Square: " << number * number << endl;
13 }

```

음수가 입력되면 제곱 계산을 건너뛰고 다음 입력으로 넘어간다. 이처럼 예외적인 경우를 먼저 처리하면 정상 처리 코드의 중첩을 줄일 수 있다.

### 실습

**파일:** 07\_continue.cpp

1부터 30까지의 정수 중 3의 배수는 건너뛰고 나머지만 출력하라.

## 9. 난수 생성의 기본 원리

숫자 맞추기 게임을 만들려면 프로그램이 정답 숫자를 생성해야 한다. 전통적인 C++ 난수 생성에는 `rand()`, `srand()`, `time()`을 사용한다.

```

1 #include <cstdlib>
2 #include <ctime>

```

### 9.1. rand()

`rand()`는 0 이상 `RAND_MAX` 이하의 정수를 반환한다.

```

1 cout << rand() << endl;
2 cout << rand() << endl;
3 cout << rand() << endl;

```

이 값들은 완전한 의미의 무작위가 아니라 정해진 계산 규칙에 따라 생성되는 의사 난수이다.

### 9.2. seed와 srand()

난수 생성기는 시작값을 기준으로 수열을 만든다. 이 시작값을 seed라고 하며, 같은 seed를 사용하면 같은 난수 수열이 생성된다.

```

1 srand(10);
2 cout << rand() << endl;

```

프로그램을 실행할 때마다 seed를 바꾸기 위해 현재 시간을 사용할 수 있다.

```

1 srand(time(0));

```

`time(0)`은 현재 시각에 따라 달라지는 값을 반환하므로 프로그램을 다시 실행할 때 다른 난수 수열이 생성될 가능성이 높다.

### 핵심 포인트

`srand(time(0))`은 일반적으로 `main()`의 시작 부분에서 한 번만 호출한다. 난수를 만들 때마다 반복 호출하면 같은 초 안에서 동일한 seed가 설정될 수 있다.

### 실습

**파일:** 08\_random.cpp

난수 다섯 개를 출력하라. 먼저 `srand()` 없이 여러 번 실행하고, 이후 `srand(time(0))`을 추가하여 결과를 비교하라.

## 10. 원하는 범위의 난수 만들기

나머지 연산자 %를 사용하면 난수의 범위를 제한할 수 있다.

```
1 rand() % 100
```

100으로 나눈 나머지는 0부터 99까지이므로 위 표현식의 결과 범위는 다음과 같다.

$$0 \leq r \leq 99$$

1부터 100까지의 값을 원하면 1을 더한다.

```
1 int number = rand() % 100 + 1;
```

$$1 \leq \text{number} \leq 100$$

### 10.1. 일반적인 범위 공식

```
1 rand() % (maximum - minimum + 1) + minimum
```

예를 들어 5부터 10까지의 정수를 생성하려면 다음과 같이 작성한다.

```
1 int number = rand() % 6 + 5;
```

| 표현식              | 생성 범위 |
|------------------|-------|
| rand() % 10      | 0-9   |
| rand() % 10 + 1  | 1-10  |
| rand() % 6 + 5   | 5-10  |
| rand() % 100 + 1 | 1-100 |

#### 핵심 포인트

rand() % N 방식은 학습용 프로그램에는 충분하지만, 엄밀하게 모든 값이 같은 확률로 생성된다고 보장되지는 않는다. 현대 C++에서는 <random> 라이브러리가 더 권장되지만, 여기서는 조건문과 반복문을 결합하는 데 집중한다.

#### 실습

**파일:** 09\_random\_range.cpp

0-9, 1-6, 10-20, -5-5 범위의 난수를 각각 생성하여 출력하라.

## 11. 종합 실습: 숫자 맞추기 게임

Day 2에서 학습한 조건문, do-while, 난수 생성을 결합하여 숫자 맞추기 게임을 작성한다.

1. 프로그램이 1부터 100 사이의 정답을 생성한다.
2. 사용자가 숫자를 입력한다.
3. 정답보다 크면 Too high!를 출력한다.
4. 정답보다 작으면 Too low!를 출력한다.
5. 정답과 같으면 Correct!를 출력한다.
6. 정답을 맞힐 때까지 반복한다.

### 11.1. 완성 코드: practice/p01\_number\_guessing.cpp

```
1 #include <iostream>
2 #include <cstdlib>
3 #include <ctime>
4
5 using namespace std;
6
7 int main()
8 {
9     srand(time(0));
10
11     int answer = rand() % 100 + 1;
12     int guess;
13     int attempts = 0;
14
15     cout << "===== " << endl;
16     cout << " Number Guessing Game " << endl;
17     cout << "===== " << endl;
18     cout << "Guess a number from 1 to 100." << endl;
19
20     do
21     {
22         cout << "Guess: ";
23         cin >> guess;
24
25         if (guess < 1 || guess > 100)
26         {
27             cout << "Enter a number from 1 to 100." << endl;
28             continue;
29         }
30
31         attempts++;
32
33         if (guess > answer)
34         {
35             cout << "Too high!" << endl;
36         }
37         else if (guess < answer)
38         {
39             cout << "Too low!" << endl;
40         }
41         else
42         {
43             cout << "Correct!" << endl;
44         }
45     }
46     while (guess != answer);
47
48     cout << "Attempts: " << attempts << endl;
49     cout << "Game Over." << endl;
50
51     return 0;
52 }
```

### 11.2. 실행 흐름 분석

정답이 68이라고 가정하자. 사용자가 50을 입력하면 `guess < answer`가 참이므로 Too low!가 출력된다. 이후 `guess != answer`가 참이므로 반복한다.

75를 입력하면 `guess > answer`가 참이므로 `Too high!`가 출력된다. 마지막으로 68을 입력하면 앞의 두 조건이 모두 거짓이고 마지막 `else`가 실행된다. 이후 반복 조건이 거짓이 되어 게임이 종료된다. 입력값이 1부터 100 사이가 아니면 `continue`가 실행된다. 따라서 정답 비교와 시도 횟수 증가는 건너뛰고 다음 입력으로 이동한다.

## 12. 프로젝트 파일 구성

```
cpp/
├── day02/
│   ├── 01_if_else.cpp
│   ├── 02_else_if.cpp
│   ├── 03_nested_if.cpp
│   ├── 04_switch.cpp
│   ├── 05_do_while.cpp
│   ├── 06_break.cpp
│   ├── 07_continue.cpp
│   ├── 08_random.cpp
│   ├── 09_random_range.cpp
│   ├── practice/
│   │   └── p01_number_guessing.cpp
│   ├── cpp_day02.tex
│   ├── cpp_day02.pdf
│   └── README.md
```

개념별 예제 파일은 Day 2 루트에 번호를 붙여 분리하고, 종합 실습 파일은 `practice` 폴더에 `p01...`, `p02...` 형식으로 저장한다.

## 13. 핵심 정리

### 핵심 포인트

- `if--else`: 두 경우 중 하나를 선택한다.
- `else if`: 여러 조건을 위에서부터 순서대로 검사한다.
- `switch`: 하나의 값과 여러 상수를 비교한다.
- `do--while`: 본문을 먼저 실행하므로 최소 한 번 실행된다.
- `break`: 가장 가까운 반복문 또는 `switch`를 종료한다.
- `continue`: 현재 반복의 남은 코드를 건너뛴다.
- `srand(time(0))`: 현재 시간을 난수 `seed`로 설정한다.
- `rand() % (max - min + 1) + min`: 지정 범위의 정수를 만든다.

## 14. 체크 질문

### 체크 질문

1. `if--else`에서 두 블록이 동시에 실행될 수 있는가?
2. `else if` 조건의 순서가 중요한 이유는 무엇인가?
3. `switch`의 각 `case`에 `break`가 없으면 어떤 일이 발생하는가?
4. `while`문과 `do--while`문의 가장 큰 차이는 무엇인가?
5. `break`와 `continue`의 차이는 무엇인가?
6. 중첩 반복문 안에서 `break`를 실행하면 몇 개의 반복문이 종료되는가?
7. `rand() % 10`의 결과 범위는 무엇인가?
8. 1부터 10까지의 난수를 만들려면 어떤 표현식을 사용하는가?
9. `srand(time(0))`은 왜 일반적으로 한 번만 호출하는가?
10. 숫자 맞추기 게임에서 `do--while`문이 적합한 이유는 무엇인가?

## 15. 추가 연습 문제

1. 정수 하나를 입력받아 절댓값을 출력하라. 표준 라이브러리의 절댓값 함수는 사용하지 않는다.
2. 서로 다른 정수 세 개를 입력받아 가장 큰 값을 출력하라.
3. 정수 두 개와 메뉴 번호를 입력받는 간단한 계산기를 `switch`문으로 작성하라. 0으로 나누는 경우도 처리한다.
4. 정답 비밀번호를 1234로 두고 올바른 값이 입력될 때까지 `do--while`문으로 반복하라.
5. 1부터 50까지 출력하되 5의 배수는 `continue`로 건너뛰어라.
6. 2 이상의 정수를 입력받고, 처음 발견한 약수를 출력한 뒤 `break`로 종료하라.
7. 1부터 6 사이의 난수를 열 번 생성하여 주사위 굴리기를 구현하라.
8. 20-30, -10-10, 100-200 범위의 난수 생성식을 각각 작성하라.
9. 숫자 맞추기 게임에 최대 시도 횟수 7회를 추가하라.
10. 게임이 끝난 뒤 `Play again? (y/n)`을 묻고, `y`이면 새 정답으로 다시 시작하도록 확장하라.